ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ, ФКСТ

Дисциплина:	„Компютърни архитектури“		
Специалност:	„КСИ“, III курс (V семестър), гр. 43	Дата:	04.11.2026
Студент:	**Йордан Начев**	Фак. №	**121224053**

Отчет по проведено лабораторно упражнение № 3

„Изследване на конвейерни компютърни архитектури (Част 2)“

Цел: Да се изследват конвейерните състезания и да се оцени доколко operand forwarding, loop unrolling, пренареждането на инструкции и jump prediction намаляват влиянието им.

1. Изследване на 5-фазен линеен конвейер

по заданието

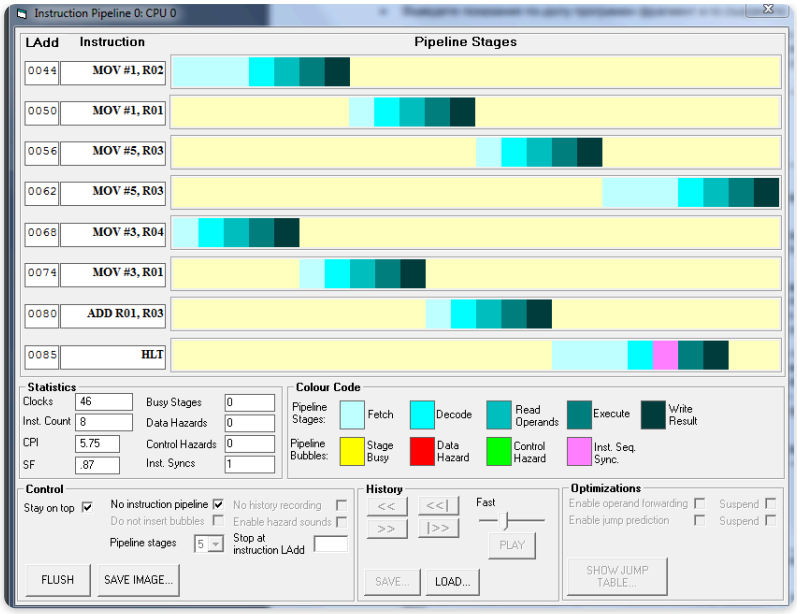
```
MOV #1, R01
MOV #5, R03
MOV #3, R01
ADD R01, R03
HLT
```

изпълнено в симулатора

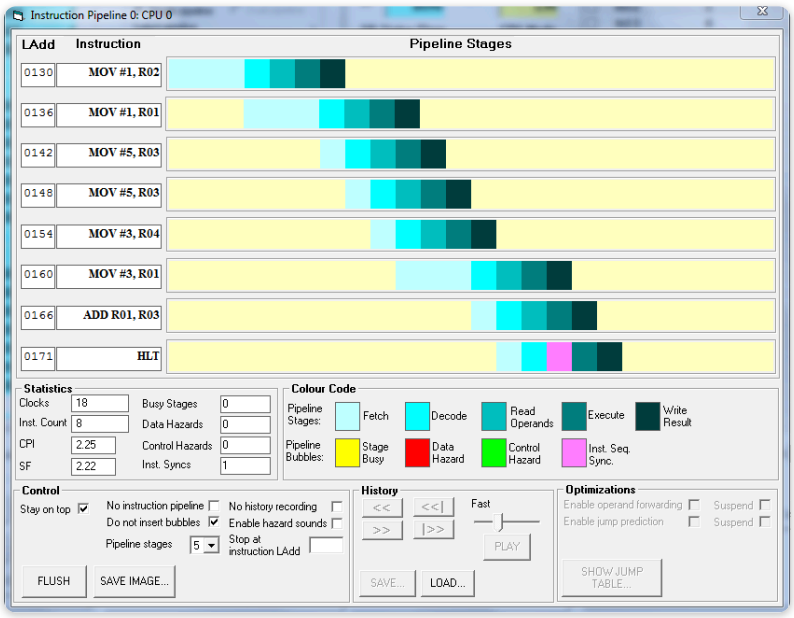
```
MOV #1, R02
MOV #1, R01
MOV #5, R03
MOV #5, R03
MOV #3, R04
MOV #3, R01
ADD R01, R03
HLT
```

При правилно изпълнение очаквам R03 = 3 + 5 = 8. Освен инструкциите от заданието програмата в симулатора съдържа MOV #1, R02, втора MOV #5, R03 и MOV #3, R04. Те не влияят на крайната стойност на R03.

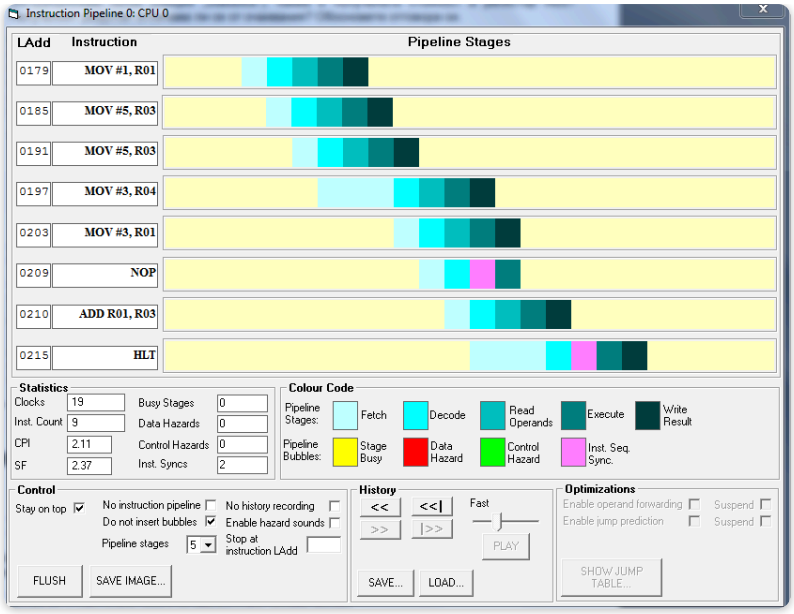
Показател	Без конвейер	Без мехури	Без мехури + NOP	С мехури
Clocks (тактове)	46	18	19	19
Inst. Count (инструкции)	8	8	9	8
CPI	5,75	2,25	2,11	2,38
SF (Speed-up Factor)	0,87	2,22	2,37	2,1
Busy Stages	0	0	0	2
Data Hazards	0	0	0	2
Control Hazards	0	0	0	0
Inst. Syncs	1	1	2	1
R03	8	**6**	8	8



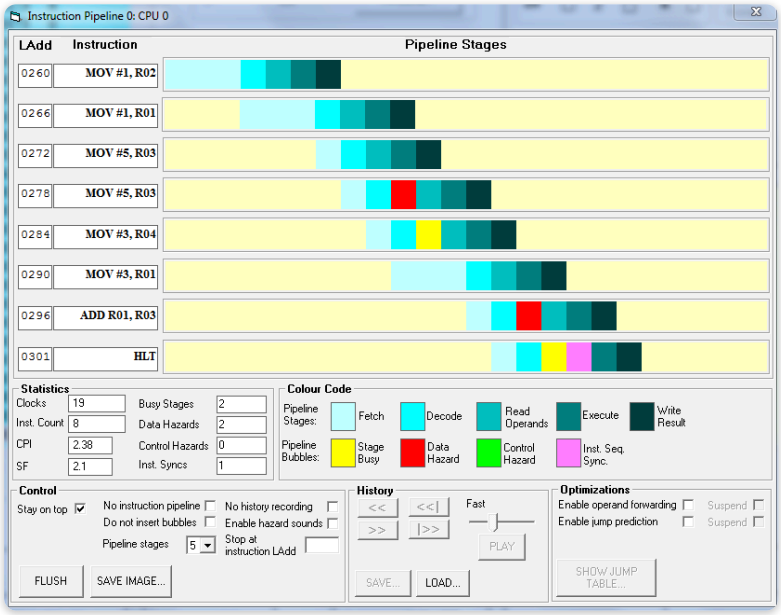
Фиг. 1. „No instruction pipeline“: последователно изпълнение



Фиг. 2. „Do not insert bubbles“: конвейер без изчакване



Фиг. 3. „Do not insert bubbles“ и вмъкната инструкция NOP



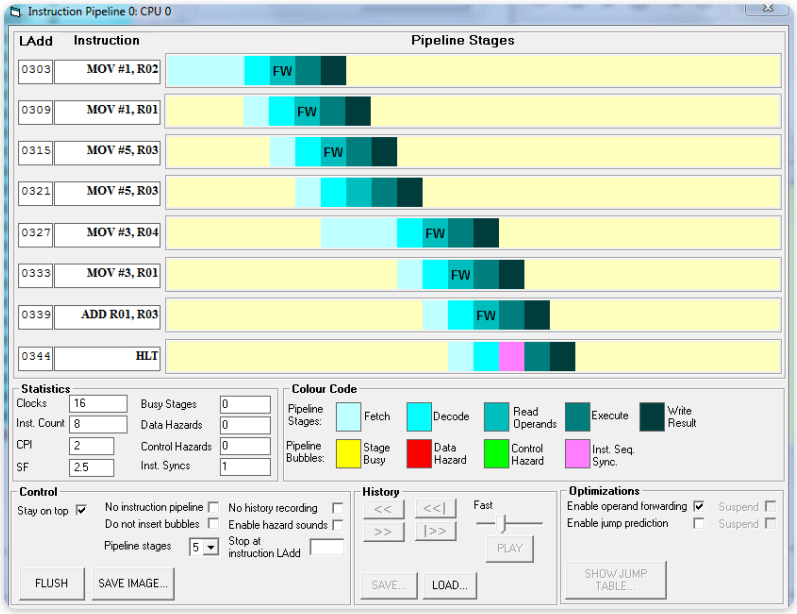
Фиг. 4. Конвейер с автоматично вмъкване на мехури

1. Последователното изпълнение отнема най-много време: 46 такта при CPI = 5,75. Стойността на R03 е 8.
2. Когато забраних мехурите, програмата завърши за 18 такта, но R03 стана 6. От времедиаграмата се вижда, че ADD чете R01 един такт преди MOV #3, R01 да запише новата стойност, така че събира 5 със старата единица.
3. След добавянето на NOP между двете инструкции четенето и записът на R01 попадат в един и същ такт и R03 отново е 8. Времето нараства до 19 такта, а инструкциите стават 9.
4. При включени мехури конвейерът сам изчаква: симулаторът отчита 2 конфликта по данни и 2 заети етапа. R03 = 8 за 19 такта, което е 2,42 пъти по-бързо от изпълнението без конвейер.

Сравнението показва, че NOP решава проблема ръчно със същата цена като автоматичните мехури.

2. Operand forwarding

Показател	С мехури	С operand forwarding
Clocks (тактове)	19	16
Inst. Count (инструкции)	8	8
CPI	2,38	2
SF (Speed-up Factor)	2,1	2,5
Busy Stages	2	0
Data Hazards	2	0
Control Hazards	0	0
Inst. Syncs	1	1



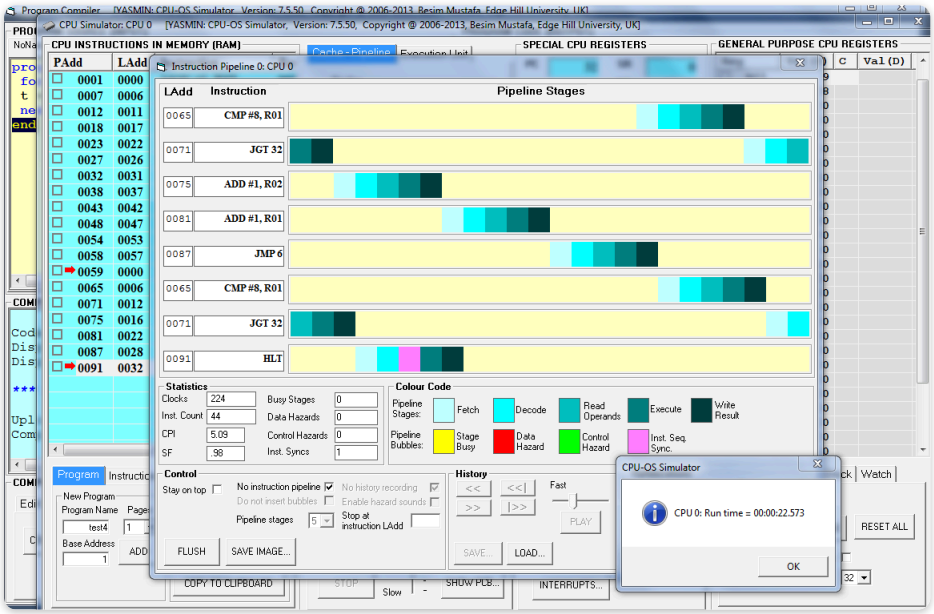
Фиг. 5. Включен operand forwarding (маркировка FW във времедиаграмата)

Operand forwarding подава резултата към зависимата инструкция веднага след фазата Execute. Така мехурите отпадат напълно: 16 такта срещу 19 (15,8% по-малко), Data Hazards и Busy Stages са 0, а CPI = 2 и SF = 2,5. Във времедиаграмата пренасочването е означено с FW. Стойността на R03 остава коректна.

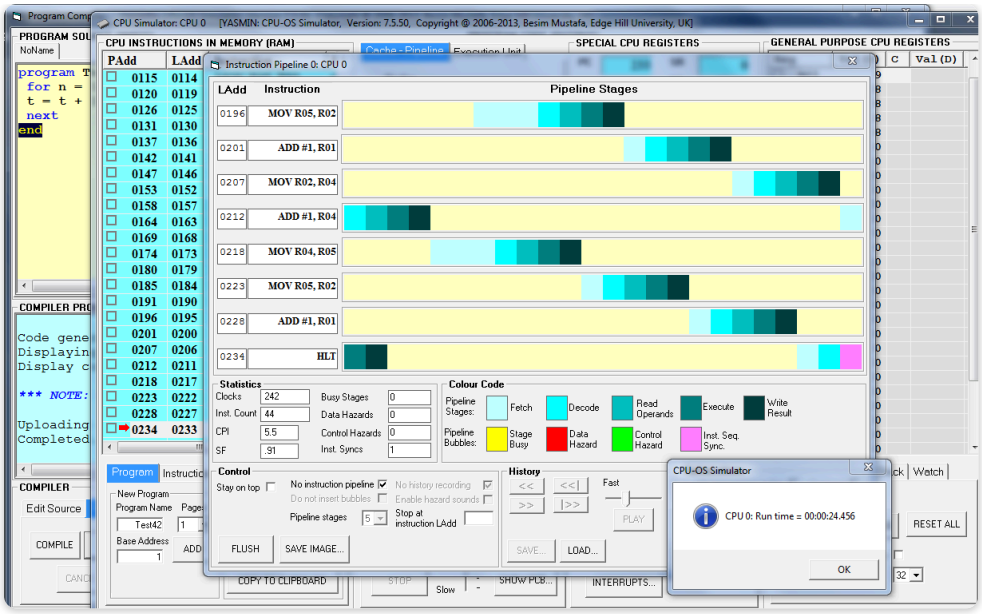
3. Loop unrolling

```
program Pipeline_Ex4_1
  for n = 1 to 8
    t = t + 1
  next
end
```

Показател	Цикъл (test4)	Loop unrolling (Test42)
Clocks (тактове)	224	242
Inst. Count (инструкции)	44	44
CPI	5,09	5,5
SF (Speed-up Factor)	0,98	0,91
Busy Stages	0	0
Data Hazards	0	0
Control Hazards	0	0
Inst. Syncs	1	1
Размер на кода, байтове	33	234
Време за изпълнение	22,573 s	24,456 s



Фиг. 6. Pipeline_Ex4_1 (в симулатора test4) с цикъл



Фиг. 7. Pipeline_Ex4_2 (в симулатора Test42) с loop unrolling

Размерът на кода нараства от 33 на 234 байта (7,1 пъти). Броят на изпълнените инструкции случайно съвпада и е 44 и в двата случая, но времето е 224 такта за цикъла и 242 за развития вариант (8% повече). CPI е 5,09 и 5,5, а SF е 0,98 и 0,91.

Важно е, че и двете стартирания са направени с отметка „No instruction pipeline“. Преходите тогава не създават конфликти (Control Hazards = 0), така че това сравнение отчита само цената на по-дългия код: при Test42 има 11 фази Fetch от по 3 такта, а при цикъла 2. Очакваната полза от loop unrolling е при работещ конвейер, където отпадат JGT и JMP.

4. Re-arranging instruction sequence

изходен код

```
program Pipleine_Ex5_1
  a = 1
  b = a
  c = 2
end
```

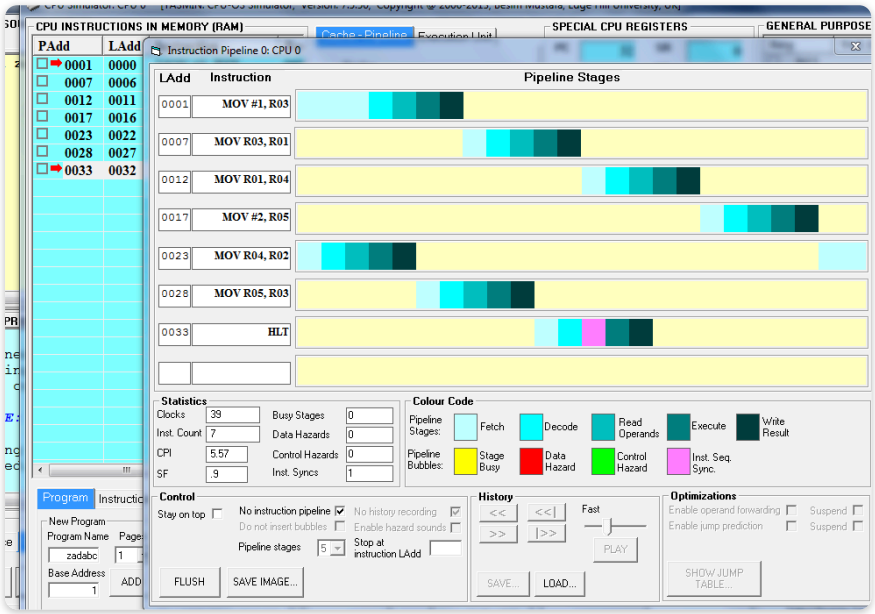
преди пренареждане

```
MOV #1, R03
MOV R03, R01
MOV R01, R04
MOV R04, R02
MOV #2, R05
MOV R05, R03
HLT
```

след пренареждане

```
MOV #1, R03
MOV R03, R01
MOV R01, R04
MOV #2, R05
MOV R04, R02
MOV R05, R03
HLT
```

Показател	Пренаредена последователност (zadabc)
Clocks (тактове)	39
Inst. Count (инструкции)	7
CPI	5,57
SF (Speed-up Factor)	0,9
Busy Stages	0
Data Hazards	0
Control Hazards	0
Inst. Syncs	1



Фиг. 8. Пренаредена инструкционна последователност, режим „No instruction pipeline“

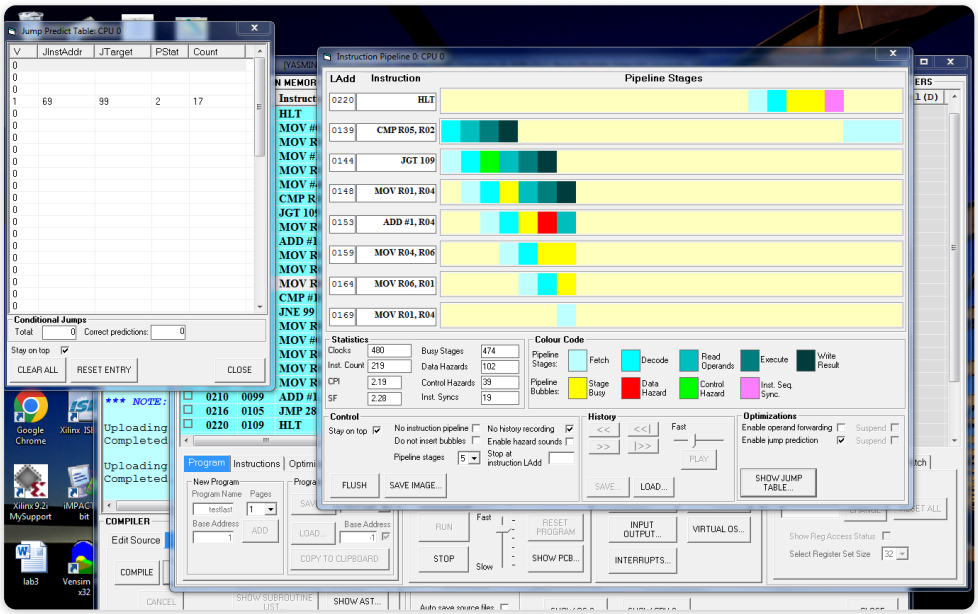
Компилаторът премества MOV #2, R05 между MOV R01, R04 и MOV R04, R02. Тази инструкция не използва регистрите на зависимата двойка, затова смисълът на програмата е същият: a = 1, b = 1, c = 2. Преместването осигурява допълнителен такт между записа и четенето на R04 и така намалява нуждата от мехури.

5. Jump prediction

```
program Pipeline_Ex6
  i = 0
  for p = 1 to 40
    i = i + 1
    if i = 10 then
      i = 0
      r = i
    end if
  next
end
```

Показател	С jump prediction
Clocks (тактове)	480
Inst. Count (инструкции)	219
CPI	2,19
SF (Speed-up Factor)	2,28
Busy Stages	474
Data Hazards	102
Control Hazards	39
Inst. Syncs	19

V	JInstAddr	JTarget	PStat	Count
1	69	99	2	17



Фиг. 9. Pipeline_Ex6 (в симулатора testlast) с jump prediction и таблицата Jump Predict Table

Таблицата Jump Predict Table пази за всеки предсказван преход дали записът е валиден (V), адреса на самия преход (JInstAddr), адреса на целта (JTarget), състоянието на предсказването (PStat) и броя обръщения (Count). При мен има един запис: 69 → 99, PStat = 2, Count = 17. Той съответства на JNE 99 от условието „if i = 10“. Този преход се извършва 9 пъти от 10, затова е удобен за предсказване.

При включено предсказване симулаторът показва 480 такта, 219 инструкции, CPI = 2,19, SF = 2,28, Data Hazards = 102 и Control Hazards = 39.

Обобщени изводи

1. **Коректност:** без изчакване зависимостта по R01 дава R03 = 6. NOP, мехурите и operand forwarding възстановяват R03 = 8.
2. **Бързодействие:** 46 такта без конвейер, 19 с мехури и 16 с operand forwarding, което е най-добрият измерен резултат.

3. **Loop unrolling:** кодът е 7,1 пъти по-голям, а при изключен конвейер времето нараства с 8%.
4. **Пренареждане и jump prediction:** пренареждането запазва логиката на програмата и раздалечава зависимите инструкции, а Jump Predict Table съхранява адреса, целта и състоянието на всеки предсказван преход.